LOVELY PROFESSIONAL UNIVERSITY

School of Computer Science and Engineering

CSES003 : EXTERNAL EXPERT INPUT - DATA STRUCTURE

Term : 25262
Roll No : R9P186A47 / Group 1
Section : R9P186A

Project: Splitwise Debt Simplifier

Optimal Expense Settlement & Animated Simulator

Table of Contents

Project: **Splitwise Debt Simplifier** | Developer: **Tuba Mirza**
INT219 – Data Structures & Algorithms | May 2026

1. Project Overview & Algorithm Design

1.1 Introduction

Settle-up networks (like Splitwise) minimize the friction of transferring money among a group of people who share multiple mutual expenses. For example, if Roommate A pays for rent, Roommate B pays for groceries, and Roommate C pays for utilities, the resulting web of debts can be convoluted. Settle-up algorithms reduce these debts to a minimal set of transactions, where people only pay what is net-owed.

1.2 The NP-Hard Problem & Subset Sum Optimality

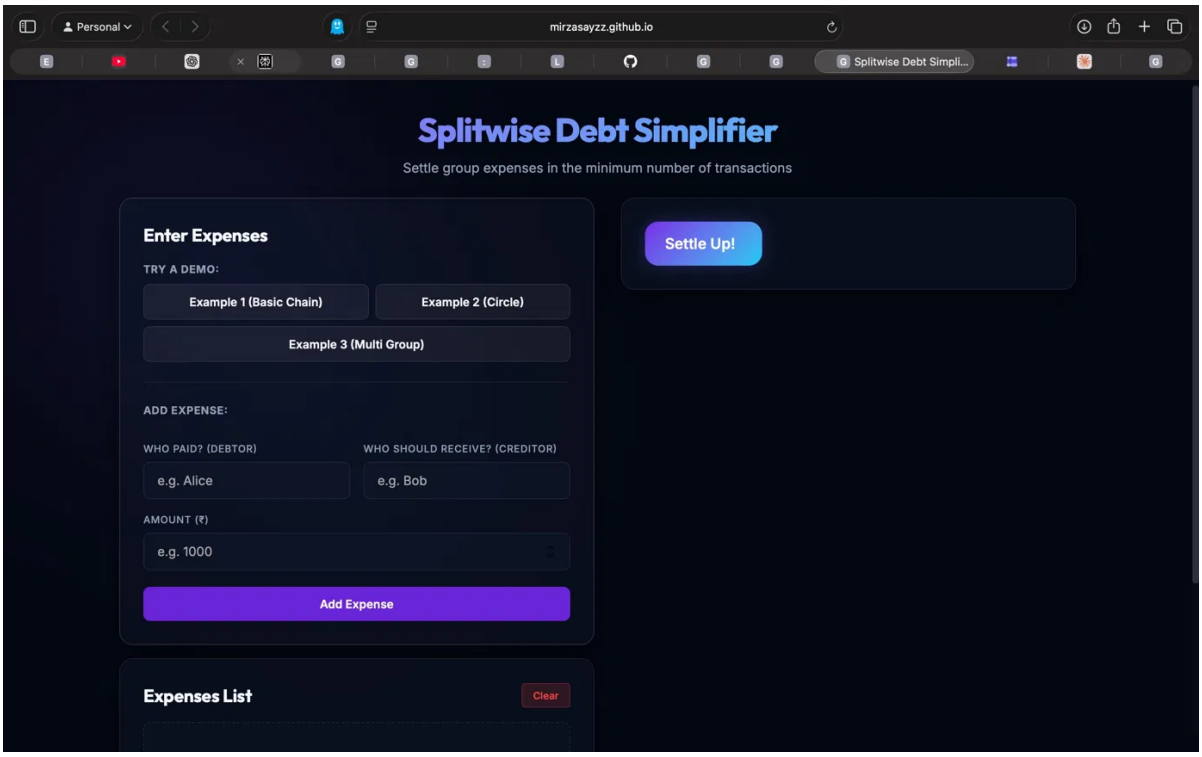
Finding the absolute minimum number of transfers is an NP-hard optimization problem. While simple greedy approaches (repeatedly matching the largest creditor with the largest debtor) can settle all balances, they do not guarantee the minimum number of transactions. For example, if A owes B Rs. 50, B owes C Rs. 50, and D owes E Rs. 10, the net balances of A, B, C, D, and E are -50, 0, +50, -10, and +10 respectively. A simple greedy solver might match them arbitrarily, but the optimal solution requires partition-matching.

Mathematically, if a group of K participants has net balances that sum to exactly 0, they can settle their debts among themselves in at most K - 1 transactions. If the entire group of N people can be partitioned into M independent zero-sum subgroups, the total transactions required will be N - M. Thus, maximizing the number of zero-sum subgroups (M) yields the minimum number of transactions.

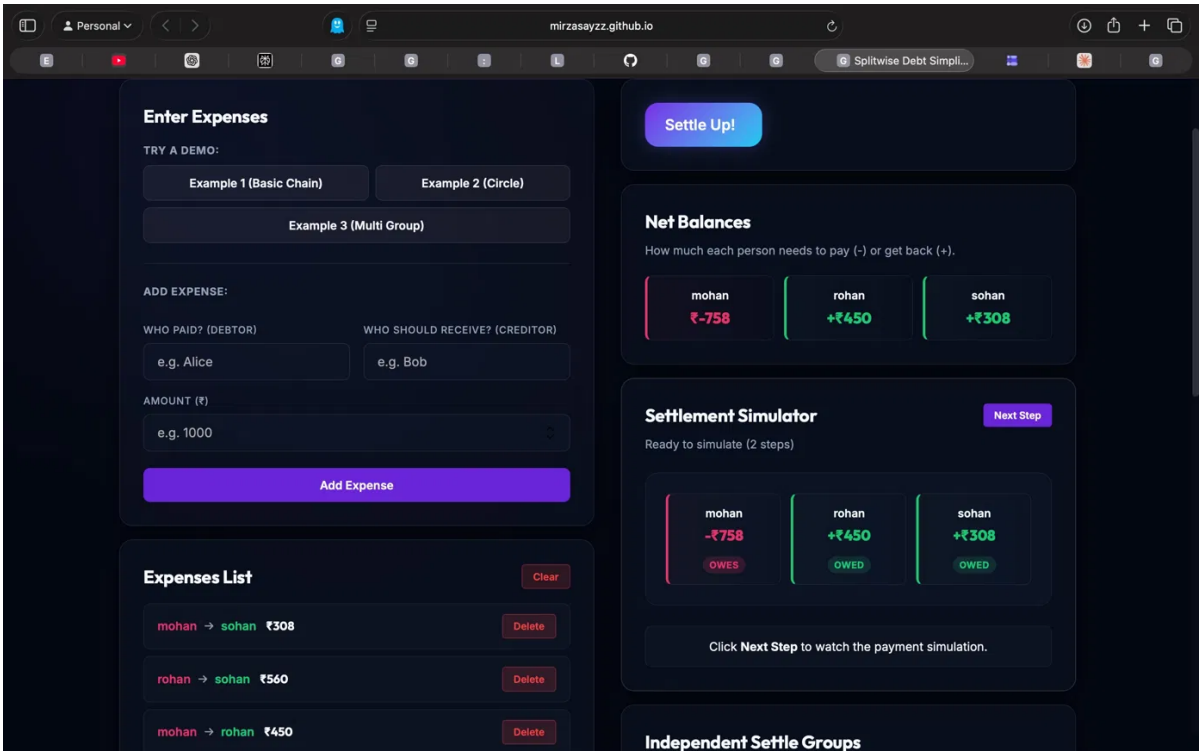
1.3 Hybrid Algorithm Implementation

To achieve maximum correctness and efficiency, this project implements a hybrid solver:
1. Optimal Solver (N <= 20): Uses a Dynamic Programming (DP) algorithm with bitmasks. It computes subset sums for all 2^N states, identifies zero-sum combinations, and uses a DP state-transition table to reconstruct the maximum number of disjoint zero-sum subgroups. The subgroups are then settled sequentially using K-1 transfers.
2. Greedy Solver Fallback (N > 20): For larger groups where DP is computationally prohibitive (due to O(2^N) complexity), the solver falls back to a Greedy priority queue (Heap) solver. It matching the largest creditor with the largest debtor in O(N log N) time, which provides a fast and highly optimized settle plan.

2. Web UI & Animated Simulator



Dashboard Interface: Input Form & Net Balances Cards



Interactive Simulator: Live Grid showing balances and settlement simulation

3. Source Code: C++ Core Optimization Library

3.1 Core Header File: include/minimize_cash_flow.h

```
#ifndef MINIMIZE_CASH_FLOW_H
#define MINIMIZE_CASH_FLOW_H

#include <vector>
#include <string>

/**
 * Simplifies a complicated network of debts to determine the minimum number of
 * transactions required to settle all balances completely.
 *
 * Each transaction in the input is represented as a vector of three strings:
 * [PersonA, PersonB, AmountStr]
 * which means PersonA owes PersonB AmountStr.
 *
 * Constraints:
 * - 1 <= number of transactions <= 10^4
 * - 1 <= amount <= 10^9
 * - Names are unique strings of English letters.
 *
 * @param transactions A list of transactions to simplify.
 * @return A minimized list of transactions in the same format.
 */
std::vector<std::vector<std::string>> minimizeCashFlow(
    std::vector<std::vector<std::string>>& transactions
);

#endif // MINIMIZE_CASH_FLOW_H
```

3.2 Core Implementation File: src/minimize_cash_flow.cpp

```
#include "minimize_cash_flow.h"
#include <unordered_map>
#include <queue>
#include <cmath>
#include <algorithm>
#include <iostream>
#include <sstream>
#include <stdexcept>

// Computes the minimized cash flow transactions
std::vector<std::vector<std::string>> minimizeCashFlow(
    std::vector<std::vector<std::string>>& transactions
) {
    // 1. Map names to unique IDs
    std::unordered_map<std::string, int> name_to_id;
    std::vector<std::string> id_to_name;

    auto get_id = [&](const std::string& name) {
        auto it = name_to_id.find(name);
        if (it != name_to_id.end()) {
            return it->second;
        }
        int id = id_to_name.size();
        name_to_id[name] = id;
        id_to_name.push_back(name);
        return id;
    };

    // Calculate net balances for each person
    // Use long long to prevent overflow (amount can be 10^9, up to 10^4 transactions)
    std::vector<long long> temp_balances;

    for (const auto& tx : transactions) {
        if (tx.size() != 3) {
```

```
            continue; // Skip invalid transactions
        }
        const std::string& debtor = tx[0];
        const std::string& creditor = tx[1];
        long long amount = std::stoll(tx[2]);
```

```
        int debtor_id = get_id(debtor);
        int creditor_id = get_id(creditor);

        if (debtor_id == creditor_id) {
            continue; // Self-debt has no effect on net balance
        }

        // Resize balances if new people were added
        if (temp_balances.size() < id_to_name.size()) {
            temp_balances.resize(id_to_name.size(), 0LL);
        }

        temp_balances[debtor_id] -= amount;
        temp_balances[creditor_id] += amount;
    }

    // Filter out people who already have a net balance of 0
    std::vector<std::string> active_names;
    std::vector<long long> active_balances;
    for (size_t i = 0; i < temp_balances.size(); ++i) {
        if (temp_balances[i] != 0) {
            active_names.push_back(id_to_name[i]);
            active_balances.push_back(temp_balances[i]);
        }
    }

    int N = active_names.size();
    std::vector<std::vector<std::string>> result;

    if (N == 0) {
        return result; // Everyone is already settled
    }

    // 2. Decide algorithm based on N
    if (N <= 20) {
        // Optimal DP with bitmask
        // dp[mask] = max number of disjoint zero-sum subsets
        int num_states = 1 << N;
        std::vector<long long> sum(num_states, 0LL);
        std::vector<int> dp(num_states, 0);
```

```
        // Precompute subset sums
        for (int mask = 1; mask < num_states; ++mask) {
            // Find lowest set bit
            int i = __builtin_ctz(mask);
            sum[mask] = sum[mask ^ (1 << i)] + active_balances[i];
        }

        // Compute DP
        for (int mask = 1; mask < num_states; ++mask) {
            int max_sub = 0;
            for (int i = 0; i < N; ++i) {
                if (mask & (1 << i)) {
                    max_sub = std::max(max_sub, dp[mask ^ (1 << i)]);
                }
            }
            dp[mask] = max_sub;
            if (sum[mask] == 0) {
                dp[mask]++;
            }
        }

        // Reconstruct components
        int active_mask = num_states - 1;
        std::vector<std::vector<int>> components;
```

```
        while (active_mask > 0) {
            if (dp[active_mask] == 1) {
                // The remaining set must form a single zero-sum component
                std::vector<int> comp;
                for (int i = 0; i < N; ++i) {
                    if (active_mask & (1 << i)) {
                        comp.push_back(i);
                    }
                }
                components.push_back(comp);
                break;
            }

            bool found = false;
```

```
            // Iterate through submasks of active_mask
            for (int T = (active_mask - 1) & active_mask; T > 0; T = (T - 1) & active_mask) {
                if (sum[T] == 0 && dp[active_mask ^ T] + 1 == dp[active_mask]) {
                    std::vector<int> comp;
                    for (int i = 0; i < N; ++i) {
                        if (T & (1 << i)) {
                            comp.push_back(i);
                        }
                    }
                    components.push_back(comp);
                    active_mask ^= T;
                    found = true;
                    break;
                }
            }

            if (!found) {
                // Fallback in case of numerical/logic edge (should not happen mathematically)
                std::vector<int> comp;
                for (int i = 0; i < N; ++i) {
                    if (active_mask & (1 << i)) {
                        comp.push_back(i);
                    }
                }
                components.push_back(comp);
                break;
            }
        }

        // Settle each component sequentially (K elements -> K - 1 transactions)
        for (const auto& comp : components) {
            int K = comp.size();
            if (K <= 1) continue;

            std::vector<long long> comp_balances(K);
            for (int i = 0; i < K; ++i) {
                comp_balances[i] = active_balances[comp[i]];
            }

            for (int i = 0; i < K - 1; ++i) {
```

```
                long long bal = comp_balances[i];
                if (bal == 0) continue;

                if (bal < 0) {
                    // Person i owes money, they pay person i+1
                    result.push_back({active_names[comp[i]], active_names[comp[i+1]], std::to_string(-bal)});
                    comp_balances[i+1] += bal;
                } else {
                    // Person i is owed money, person i+1 pays person i
                    result.push_back({active_names[comp[i+1]], active_names[comp[i]], std::to_string(bal)});
                    comp_balances[i+1] += bal;
                }
                comp_balances[i] = 0;
            }
        }

    } else {
        // Fallback: Greedy algorithm using Max-Heap & Min-Heap
```

```
        std::priority_queue<std::pair<long long, int>> max_heap; // {balance, person_idx}
        // Min-heap for negative balances (storing as negative values, so standard min-heap logic works)
        std::priority_queue<std::pair<long long, int>,
                            std::vector<std::pair<long long, int>>,
                            std::greater<std::pair<long long, int>>> min_heap;

        for (int i = 0; i < N; ++i) {
            if (active_balances[i] > 0) {
                max_heap.push({active_balances[i], i});
            } else if (active_balances[i] < 0) {
                min_heap.push({active_balances[i], i});
            }
        }

        while (!max_heap.empty() && !min_heap.empty()) {
            auto credit = max_heap.top();
            max_heap.pop();
            auto debit = min_heap.top();
            min_heap.pop();

            long long credit_val = credit.first;
            long long debit_val = -debit.first;
            long long settle_val = std::min(credit_val, debit_val);

            // Debtor pays Creditor
            result.push_back({active_names[debit.second], active_names[credit.second], std::to_string(settle_val)});

            long long rem_credit = credit_val - settle_val;
            long long rem_debit = debit.first + settle_val;

            if (rem_credit > 0) {
                max_heap.push({rem_credit, credit.second});
            }
            if (rem_debit < 0) {
                min_heap.push({rem_debit, debit.second});
            }
        }
    }

    return result;
}
```

4. Source Code: CLI Wrapper & Web Controller

4.1 Command Line Entrypoint: src/main.cpp

```
#include "minimize_cash_flow.h"
#include <iostream>
#include <vector>
#include <string>
#include <iomanip>

void printTransactions(const std::vector<std::vector<std::string>>& txs) {
    if (txs.empty()) {
        std::cout << "  No transactions needed (already settled)!\n";
        return;
    }
    for (const auto& tx : txs) {
        std::cout << "  " << tx[0] << " pays " << tx[1] << " \u20b9" << tx[2] << "\n";
    }
}

void runDemo(const std::string& title, std::vector<std::vector<std::string>> txs) {
    std::cout << "\n==================================================\n";
    std::cout << " RUNNING DEMO: " << title << "\n";
    std::cout << "==================================================\n";
    std::cout << "Original Transactions:\n";
    for (const auto& tx : txs) {
        std::cout << "  " << tx[0] << " owes " << tx[1] << " \u20b9" << tx[2] << "\n";
    }

    auto optimized = minimizeCashFlow(txs);
    std::cout << "\nOptimized Settlement (" << optimized.size() << " transactions):\n";
    printTransactions(optimized);
}

int main(int argc, char* argv[]) {
    // Enable UTF-8 console output for currency symbol if supported
    #ifdef _WIN32
    // Windows specific setup if needed, but we are on Mac
    #endif

    std::cout << "==================================================\n";
    std::cout << "   DEBT SIMPLIFICATION / CASH FLOW MINIMIZER      \n";
    std::cout << "==================================================\n";
```

```
    if (argc > 1 && std::string(argv[1]) == "--interactive") {
        std::cout << "\n[Interactive Mode]\n";
        std::cout << "Enter the number of transactions: ";
        int n;
        if (!(std::cin >> n) || n <= 0) {
            std::cerr << "Invalid number of transactions.\n";
            return 1;
        }

        std::vector<std::vector<std::string>> transactions;
        std::cout << "Enter each transaction as: Debtor Creditor Amount\n";
        std::cout << "Example: Alice Bob 1000\n";
        for (int i = 0; i < n; ++i) {
            std::string debtor, creditor, amount_str;
            std::cout << "Transaction #" << i + 1 << ": ";
            std::cin >> debtor >> creditor >> amount_str;
            transactions.push_back({debtor, creditor, amount_str});
        }

        std::cout << "\nSimplifying debts...\n";
        auto optimized = minimizeCashFlow(transactions);
        std::cout << "\nOptimized Transactions (" << optimized.size() << " transfers):\n";
        printTransactions(optimized);
        return 0;
    }
```

```
    // Default mode: Run demos
    // 1. Example 1
    runDemo("Example 1 (Chain Settle)", {
        {"Tom", "Jerry", "1000"},
        {"Jerry", "Spike", "1000"},
        {"Spike", "Tom", "500"}
    });

    // 2. Example 2
    runDemo("Example 2 (Circular Debts)", {
        {"Alice", "Bob", "4000"},
        {"Bob", "Charlie", "2000"},
        {"Charlie", "David", "1000"},
        {"David", "Alice", "500"}
    });

    // 3. DP vs Greedy Comparison Demo
    // Balances:
    // Person0 (A): +9
    // Person1 (B): -5
    // Person2 (C): -4
    // Person3 (D): +6
    // Person4 (E): -3
    // Person5 (F): -3
    // Under Greedy: takes 5 transactions.
    // Under DP (Optimal): splits into {A, B, C} and {D, E, F}, taking 4 transactions.
    runDemo("Greedy vs DP Optimality Comparison", {
        {"B", "A", "5"},
        {"C", "A", "4"},
        {"E", "D", "3"},
        {"F", "D", "3"}
    });

    std::cout << "\nTo run interactive mode, execute: ./minimize_cash_flow --interactive\n\n";
    return 0;
}
```

4.2 JavaScript Controller (Core Simulator section): app.js

```
function simplifyDebts() {
    // Clear logs
    logContainer.innerHTML = '';
    logStep("Starting Debt Simplification...", "highlight");

    // 1. Map names to unique IDs
    const nameToId = new Map();
    const idToName = [];

    function getId(name) {
        if (nameToId.has(name)) {
            return nameToId.get(name);
        }
        const id = idToName.length;
        nameToId.set(name, id);
        idToName.push(name);
        logStep(`  - Registered: ${name} (Index ID: ${id})`, 'info');
        return id;
    }

    logStep("Step 1: Mapping participant names to unique indices...", "highlight");

    // Calculate net balances
    const tempBalances = [];
    currentTransactions.forEach(tx => {
        const dId = getId(tx.debtor);
        const cId = getId(tx.creditor);

        while (tempBalances.length < idToName.length) {
            tempBalances.push(0);
        }
```

```
        tempBalances[dId] -= tx.amount;
        tempBalances[cId] += tx.amount;
    });

    logStep("Step 2: Processing transactions to calculate initial net balances...", "highlight");
    currentTransactions.forEach(tx => {
              logStep(`    -  ${tx.debtor}  owes  ${tx.creditor}  Rs.${tx.amount}  ->  ${tx.debtor}'s  balance  decreases,
${tx.creditor}'s balance increases.`, 'info');
    });
```

```
    logStep("Computed Initial Net Balances:", "highlight");
    idToName.forEach((name, id) => {
        const bal = tempBalances[id];
        if (bal < 0) {
            logStep(`  - ${name}: -Rs.${-bal} (needs to pay)`, 'warning');
        } else if (bal > 0) {
            logStep(`  - ${name}: +Rs.${bal} (needs to receive)`, 'success');
        } else {
            logStep(`  - ${name}: Rs.0 (already settled)`, 'info');
        }
    });

    // Filter non-zero active members
    logStep("Step 3: Filtering out members who are already settled (net balance of 0)...", "highlight");
    const activeNames = [];
    const activeBalances = [];

    tempBalances.forEach((bal, id) => {
        if (bal !== 0) {
            activeNames.push(idToName[id]);
            activeBalances.push(bal);
        } else {
            logStep(`  - ${idToName[id]} has a net balance of 0. Filtered out from calculations.`, 'info');
        }
    });

    const N = activeNames.length;
    logStep(`Active participants to settle: ${N} (${activeNames.join(', ')})`, 'info');

    // Display balances
    renderBalancesUI(activeNames, activeBalances);

    if (N === 0) {
        logStep("All members are already settled! No transfers needed.", "success");
        renderTransfersUI([]);
        sectionLog.style.display = 'block';
        return;
    }
```

```
    let simplifiedTransfers = [];

    if (N <= 20) {
        logStep(`Active member count is N = ${N}. Running optimal grouping solver.`, 'highlight');

        const numStates = 1 << N;
        const sum = new Array(numStates).fill(0);
        const dp = new Array(numStates).fill(0);

        logStep(`Scanning balance combinations to find independent groups...`, 'info');
        // Precompute subset sums
        for (let mask = 1; mask < numStates; ++mask) {
            const i = 31 - Math.clz32(mask & -mask);
            sum[mask] = sum[mask ^ (1 << i)] + activeBalances[i];
        }

        logStep(`Calculating maximum possible zero-sum subsets...`, 'info');
        // Compute DP
        for (let mask = 1; mask < numStates; ++mask) {
            let maxSub = 0;
            for (let i = 0; i < N; ++i) {
                if (mask & (1 << i)) {
                    maxSub = Math.max(maxSub, dp[mask ^ (1 << i)]);
```

```
                }
            }
            dp[mask] = maxSub;
            if (sum[mask] === 0) {
                dp[mask]++;
            }
        }

        const maxZeroSumSubsets = dp[numStates - 1];
        logStep(`Found ${maxZeroSumSubsets} independent settle groups.`, 'success');
             logStep(`Transactions needed: N - Groups = ${N} - ${maxZeroSumSubsets} = ${N - maxZeroSumSubsets}`,
'highlight');

        logStep(`Step 4: Isolating the independent groups...`, 'highlight');
        let activeMask = numStates - 1;
        const components = [];

        while (activeMask > 0) {
```

```
            if (dp[activeMask] === 1) {
                const comp = [];
                for (let i = 0; i < N; ++i) {
                    if (activeMask & (1 << i)) {
                        comp.push(i);
                    }
                }
                components.push(comp);
                logStep(`  - Settle Group: [${comp.map(idx => activeNames[idx]).join(', ')}] (sum is 0)`, 'code');
                break;
            }

            let found = false;
            for (let T = (activeMask - 1) & activeMask; T > 0; T = (T - 1) & activeMask) {
                if (sum[T] === 0 && dp[activeMask ^ T] + 1 === dp[activeMask]) {
                    const comp = [];
                    for (let i = 0; i < N; ++i) {
                        if (T & (1 << i)) {
                            comp.push(i);
                        }
                    }
                    components.push(comp);
                    logStep(`  - Settle Group: [${comp.map(idx => activeNames[idx]).join(', ')}] (sum is 0)`, 'code');
                    activeMask ^= T;
                    found = true;
                    break;
                }
            }

            if (!found) {
                const comp = [];
                for (let i = 0; i < N; ++i) {
                    if (activeMask & (1 << i)) {
                        comp.push(i);
                    }
                }
                components.push(comp);
                logStep(`  - Settle Group: [${comp.map(idx => activeNames[idx]).join(', ')}] (sum is 0)`, 'code');
                break;
            }
```

```
        }

        // Render zero-sum subsets
        renderComponentsUI(components, activeNames);

        // Settle components sequentially
        logStep(`Step 5: Settling each group (K members need K-1 transfers)...`, 'highlight');
        components.forEach((comp, compIdx) => {
            const K = comp.length;
            if (K <= 1) return;

            logStep(`Settling Group #${compIdx + 1} (${comp.map(idx => activeNames[idx]).join(', ')}):`, 'highlight');
            const compBalances = comp.map(idx => activeBalances[idx]);
```

```
            for (let i = 0; i < K - 1; ++i) {
                const bal = compBalances[i];
                if (bal === 0) {
                    logStep(`  - ${activeNames[comp[i]]} is already settled (0 balance).`, 'info');
                    continue;
                }

                if (bal < 0) {
                                    logStep(`    -  ${activeNames[comp[i]]}  has  a  debt  of  Rs.${-bal}.  They  pay
${activeNames[comp[i+1]]}.`, 'warning');
                    simplifiedTransfers.push({
                        from: activeNames[comp[i]],
                        to: activeNames[comp[i+1]],
                        amount: -bal
                    });
                    compBalances[i+1] += bal;
                    logStep(`    - Balance of ${activeNames[comp[i+1]]} updated to Rs.${compBalances[i+1]}.`, 'info');
                } else {
                       logStep(`   - ${activeNames[comp[i]]} is owed Rs.${bal}. ${activeNames[comp[i+1]]} pays them.`,
'success');
                    simplifiedTransfers.push({
                        from: activeNames[comp[i+1]],
                        to: activeNames[comp[i]],
                        amount: bal
                    });
                    compBalances[i+1] += bal;
                    logStep(`    - Balance of ${activeNames[comp[i+1]]} updated to Rs.${compBalances[i+1]}.`, 'info');
                }
                compBalances[i] = 0;
            }
            logStep(`  - Group #${compIdx + 1} is now fully settled!`, 'success');
        });

    } else {
        logStep(`Active members N = ${N} (> 20). Running greedy settlement solver.`, 'highlight');

        const maxHeap = new PriorityQueue((a, b) => a.val > b.val);
        const minHeap = new PriorityQueue((a, b) => a.val < b.val);

        logStep(`Loading creditors and debtors into priority queues...`, 'info');
        for (let i = 0; i < N; ++i) {
            if (activeBalances[i] > 0) {
                maxHeap.push({ val: activeBalances[i], idx: i });
                logStep(`  - Creditor: ${activeNames[i]} (+Rs.${activeBalances[i]})`, 'success');
            } else if (activeBalances[i] < 0) {
                minHeap.push({ val: activeBalances[i], idx: i });
                logStep(`  - Debtor: ${activeNames[i]} (-Rs.${-activeBalances[i]})`, 'warning');
            }
        }

        let step = 1;
        while (!maxHeap.isEmpty() && !minHeap.isEmpty()) {
            const credit = maxHeap.pop();
            const debit = minHeap.pop();

            const creditVal = credit.val;
            const debitVal = -debit.val;
            const settleVal = Math.min(creditVal, debitVal);

            logStep(`Greedy Match #${step++}:`, 'highlight');
            logStep(`  - Max creditor: ${activeNames[credit.idx]} (+Rs.${creditVal})`, 'success');
            logStep(`  - Max debtor: ${activeNames[debit.idx]} (-Rs.${debitVal})`, 'warning');
               logStep(`   - Settlement: ${activeNames[debit.idx]} pays ${activeNames[credit.idx]} Rs.${settleVal}.`,
'info');

            simplifiedTransfers.push({
                from: activeNames[debit.idx],
                to: activeNames[credit.idx],
                amount: settleVal
            });

            const remCredit = creditVal - settleVal;
```

```
            const remDebit = debit.val + settleVal;

            if (remCredit > 0) {
                maxHeap.push({ val: remCredit, idx: credit.idx });
                    logStep(`     - ${activeNames[credit.idx]} still owed Rs.${remCredit}. Pushed back to Max-Heap.`,
'info');
            } else {
                logStep(`    - ${activeNames[credit.idx]} is now fully settled!`, 'success');
            }
            if (remDebit < 0) {
                minHeap.push({ val: remDebit, idx: debit.idx });
                    logStep(`     - ${activeNames[debit.idx]} still owes Rs.${-remDebit}. Pushed back to Min-Heap.`,
'info');
            } else {
                logStep(`    - ${activeNames[debit.idx]} is now fully settled!`, 'success');
            }
        }
    }

    sectionLog.style.display = 'block';
    renderTransfersUI(simplifiedTransfers);
    initSimulator(simplifiedTransfers, activeNames, activeBalances);
}

function renderBalancesUI(names, balances) {
    balancesGrid.innerHTML = '';

    // Sort names alphabetically so the list is stable
    const list = names.map((name, i) => ({ name, bal: balances[i] }))
                      .sort((a, b) => a.name.localeCompare(b.name));

    list.forEach(item => {
        const div = document.createElement('div');
        div.className = `balance-card ${item.bal < 0 ? 'balance-debtor' : 'balance-creditor'}`;
        div.innerHTML = `
            <div class="balance-name" title="${item.name}">${item.name}</div>
            <div class="balance-val">${item.bal > 0 ? '+' : ''}Rs.${item.bal}</div>
        `;
        balancesGrid.appendChild(div);
    });

    sectionBalances.style.display = 'block';
}

function renderComponentsUI(components, activeNames) {
    componentsContainer.innerHTML = '';

    components.forEach((comp, idx) => {
        const div = document.createElement('div');
        div.className = 'component-card';

        const badgesHtml = comp.map(memberIdx =>
            `<span class="component-member-badge">${activeNames[memberIdx]}</span>`
        ).join('');

        div.innerHTML = `
            <div class="component-title">Group #${idx + 1}</div>
            <div class="component-members">${badgesHtml}</div>
        `;
        componentsContainer.appendChild(div);
    });

    sectionComponents.style.display = 'block';
}

function renderTransfersUI(transfers) {
    transfersList.innerHTML = '';
    transfersBadge.textContent = `${transfers.length} transfer${transfers.length === 1 ? '' : 's'}`;

    if (transfers.length === 0) {
        transfersList.innerHTML = `<div class="empty-state">Everyone is fully settled! No transfers needed.</div>`;
    } else {
        transfers.forEach(tf => {
```

```
            const li = document.createElement('li');
            li.className = 'transfer-item';
            li.innerHTML = `
                <span class="transfer-details">
                    <span class="transfer-payee">${tf.from}</span>
                    <span class="transfer-middle">pays</span>
                    <span class="transfer-rec">${tf.to}</span>
                </span>
                <span class="transfer-val">Rs.${tf.amount}</span>
            `;
            transfersList.appendChild(li);
        });
    }

    sectionTransfers.style.display = 'block';
}

function initSimulator(transfers, activeNames, activeBalances) {
    simTransfers = transfers;
    simActiveNames = [...activeNames];
    simActiveBalances = [...activeBalances];
    simRunningBalances = [...activeBalances];
    simCurrentStep = 0;

    if (transfers.length === 0) {
        sectionSimulator.style.display = 'none';
        return;
    }

    btnSimAction.textContent = 'Next Step';
    btnSimAction.disabled = false;
    sectionSimulator.style.display = 'block';

    simStepText.textContent = `Ready to simulate (${transfers.length} step${transfers.length === 1 ? '' : 's'})`;
    simExplanation.innerHTML = `Click <strong>Next Step</strong> to watch the payment simulation.`;

    renderSimulationGrid();
}

function renderSimulationGrid() {
    simGrid.innerHTML = '';

    // Sort names alphabetically to keep card layout stable and consistent
    const sortedActive = simActiveNames.map((name, idx) => ({
        name,
        idx
    })).sort((a, b) => a.name.localeCompare(b.name));

    sortedActive.forEach(item => {
        const bal = simRunningBalances[item.idx];
        const card = document.createElement('div');
        card.className = 'sim-card';
        card.setAttribute('data-name', item.name);

        let colorClass = 'sim-card-settled';
        let displayBal = 'Rs.0';
        let badgeHtml = '<span class="sim-card-badge settled-badge">Settled [check]</span>';

        if (bal < 0) {
            colorClass = 'sim-card-debtor';
            displayBal = `-Rs.${-bal}`;
            badgeHtml = '<span class="sim-card-badge debtor-badge">Owes</span>';
        } else if (bal > 0) {
            colorClass = 'sim-card-creditor';
            displayBal = `+Rs.${bal}`;
            badgeHtml = '<span class="sim-card-badge creditor-badge">Owed</span>';
        }

        card.classList.add(colorClass);
        card.innerHTML = `
            <div class="sim-card-title" title="${item.name}">${item.name}</div>
            <div class="sim-card-val">${displayBal}</div>
```

```
            <div class="sim-card-status">${badgeHtml}</div>
        `;

        simGrid.appendChild(card);
    });
}

function updateCardVisual(name, newBalance) {
    const card = document.querySelector(`.sim-card[data-name="${name}"]`);
    if (!card) return;

    // Reset all status classes
    card.classList.remove('sim-card-debtor', 'sim-card-creditor', 'sim-card-settled');
```

```
    const valEl = card.querySelector('.sim-card-val');
    const statusEl = card.querySelector('.sim-card-status');

    let colorClass = 'sim-card-settled';
    let displayBal = 'Rs.0';
    let badgeHtml = '<span class="sim-card-badge settled-badge">Settled [check]</span>';

    if (newBalance < 0) {
        colorClass = 'sim-card-debtor';
        displayBal = `-Rs.${-newBalance}`;
        badgeHtml = '<span class="sim-card-badge debtor-badge">Owes</span>';
    } else if (newBalance > 0) {
        colorClass = 'sim-card-creditor';
        displayBal = `+Rs.${newBalance}`;
        badgeHtml = '<span class="sim-card-badge creditor-badge">Owed</span>';
    }

    card.classList.add(colorClass);
    if (valEl) valEl.textContent = displayBal;
    if (statusEl) statusEl.innerHTML = badgeHtml;
}

function renderSimulationStep() {
    if (simTransfers.length === 0 || simCurrentStep >= simTransfers.length) return;

    const tx = simTransfers[simCurrentStep];
    simStepText.textContent = `Step ${simCurrentStep + 1} of ${simTransfers.length}`;

    // Disable action button to prevent click spamming during animation
    btnSimAction.disabled = true;

    // Remove any previous highlights
    document.querySelectorAll('.sim-card').forEach(card => {
        card.classList.remove('sim-active-debtor', 'sim-active-creditor');
    });

    const debtorCard = document.querySelector(`.sim-card[data-name="${tx.from}"]`);
    const creditorCard = document.querySelector(`.sim-card[data-name="${tx.to}"]`);

    if (!debtorCard || !creditorCard) {
```

```
        // Fallback if cards not found
        btnSimAction.disabled = false;
        return;
    }

    debtorCard.classList.add('sim-active-debtor');
    creditorCard.classList.add('sim-active-creditor');

    // Get positions relative to .simulator-view container
    const viewRect = simulatorView.getBoundingClientRect();
    const debtorRect = debtorCard.getBoundingClientRect();
    const creditorRect = creditorCard.getBoundingClientRect();

    const startX = debtorRect.left - viewRect.left + debtorRect.width / 2;
    const startY = debtorRect.top - viewRect.top + debtorRect.height / 2;
    const endX = creditorRect.left - viewRect.left + creditorRect.width / 2;
    const endY = creditorRect.top - viewRect.top + creditorRect.height / 2;
```

```
    // Create the flying coin element
    const coin = document.createElement('div');
    coin.className = 'flying-coin';
    coin.textContent = '[money]';
    coin.style.left = startX + 'px';
    coin.style.top = startY + 'px';
    simulatorView.appendChild(coin);

      simExplanation.innerHTML = `[money] <strong>${tx.from}</strong> is sending <strong>Rs.${tx.amount}</strong> to
<strong>${tx.to}</strong>...`;

    // Trigger transition after rendering
    requestAnimationFrame(() => {
        requestAnimationFrame(() => {
            coin.style.left = endX + 'px';
            coin.style.top = endY + 'px';
        });
    });

    // Update balances and card state at the end of the transition (850ms)
    setTimeout(() => {
        // Remove coin element
        coin.remove();
```

```
        const debtorIdx = simActiveNames.indexOf(tx.from);
        const creditorIdx = simActiveNames.indexOf(tx.to);

        const oldDebtorBal = simRunningBalances[debtorIdx];
        const oldCreditorBal = simRunningBalances[creditorIdx];

        simRunningBalances[debtorIdx] += tx.amount;
        simRunningBalances[creditorIdx] -= tx.amount;

        const newDebtorBal = simRunningBalances[debtorIdx];
        const newCreditorBal = simRunningBalances[creditorIdx];

        // Update card visual elements in place
        updateCardVisual(tx.from, newDebtorBal);
        updateCardVisual(tx.to, newCreditorBal);

        // Build status details
                    let   outcomeMsg   =   `<strong>${tx.from}</strong>   paid   <strong>Rs.${tx.amount}</strong>   to
<strong>${tx.to}</strong>!<br><small>`;

        if (newDebtorBal === 0) {
            outcomeMsg += `[success] <strong>${tx.from}</strong> is now fully settled (Rs.0 balance)!<br>`;
        } else {
                         outcomeMsg  +=  `*  <strong>${tx.from}</strong>'s  remaining  debt:  Rs.${-oldDebtorBal}  ->
Rs.${-newDebtorBal}.<br>`;
        }

        if (newCreditorBal === 0) {
            outcomeMsg += `[success] <strong>${tx.to}</strong> is now fully paid (Rs.0 balance)!`;
        } else {
                        outcomeMsg  +=  `*  <strong>${tx.to}</strong>'s  remaining  credit:  +Rs.${oldCreditorBal}  ->
+Rs.${newCreditorBal}.`;
        }
        outcomeMsg += `</small>`;

        simExplanation.innerHTML = outcomeMsg;
        btnSimAction.disabled = false;

        simCurrentStep++;
        if (simCurrentStep >= simTransfers.length) {
            btnSimAction.textContent = 'Reset Simulation';
             simExplanation.innerHTML += `<br><strong>All debts are fully settled!</strong> <small>Click Reset to watch
again.</small>`;
```

```
        }
    }, 850);
}

// Single Action Button click handler
```

```
btnSimAction.addEventListener('click', () => {
    if (simTransfers.length === 0) return;

    if (btnSimAction.textContent === 'Reset Simulation') {
        simCurrentStep = 0;
        simRunningBalances = [...simActiveBalances];
        btnSimAction.textContent = 'Next Step';

        // Remove highlights
        document.querySelectorAll('.sim-card').forEach(card => {
            card.classList.remove('sim-active-debtor', 'sim-active-creditor');
        });

        renderSimulationGrid();

          simStepText.textContent = `Ready to simulate (${simTransfers.length} step${simTransfers.length === 1 ? '' :
's'})`;
        simExplanation.innerHTML = `Click <strong>Next Step</strong> to watch the payment simulation.`;
        return;
    }

    renderSimulationStep();
});
```